

MIGUEL ANGEL SALAZAR MARTINEZ

Universidad Politécnica de San Luis Potosi

CNO 4 – Seguridad Informática

Profesor: Servando López Contreras

Actividad 7 – File Path Traversal Simple Case

18/09/26

Introducción

En la actualidad, el desarrollo de aplicaciones web se ha convertido en el pilar fundamental de la mayoría de los servicios digitales que consumimos a diario. Sin embargo, junto con la evolución de la tecnología, también han evolucionado las técnicas que utilizan los ciberdelincuentes para vulnerar estos sistemas. Una de las vulnerabilidades más críticas y, a menudo, subestimadas es el File Path Traversal (también conocido como Directory Traversal). Comprender y saber explotar esta vulnerabilidad no es un fin en sí mismo, sino una herramienta fundamental para que los desarrolladores y profesionales de la ciberseguridad puedan entender cómo piensa un atacante y, de esta manera, construir defensas sólidas.

La importancia de saber usar herramientas como Burp Suite para detectar y explotar el Path Traversal radica en la necesidad de proteger la integridad de los sistemas. Un atacante que domina esta técnica puede acceder a archivos críticos del servidor, como configuraciones, contraseñas o datos de usuarios, lo que pone en riesgo total la confidencialidad, integridad y disponibilidad de la información (la tríada CIA). Al entender cómo funciona el uso de secuencias como `../` para navegar fuera del directorio raíz web, los defensores pueden implementar validaciones de entrada estrictas, listas blancas y configuraciones de servidor adecuadas. En resumen, estudiar el Path Traversal es una lección esencial de defensa: para proteger una fortaleza, primero hay que saber cómo un enemigo podría escalar sus muros.

Marco Teórico

El File Path Traversal (o Directory Traversal) es una vulnerabilidad de seguridad web que permite a un atacante leer archivos arbitrarios en el servidor que ejecuta una aplicación. Esta falla ocurre cuando una aplicación web utiliza datos proporcionados por el usuario para construir una ruta de archivo, sin la debida sanitización o validación. El atacante explota esta confianza para "viajar" fuera del directorio previsto (generalmente el directorio web) y acceder a archivos del sistema operativo que no deberían estar expuestos.

El núcleo del ataque reside en el uso de secuencias de puntos y barras (../). En los sistemas operativos basados en Unix/Linux, el símbolo .. representa el directorio padre. Al concatenar múltiples instancias de ../, el atacante puede subir en la jerarquía de directorios del servidor. Por ejemplo, si una aplicación web solicita una imagen mediante filename=imagen.jpg y el código del servidor hace `readfile("/var/www/images/" . $_GET['filename'])`, el atacante puede modificar la petición a filename=../../etc/passwd. El servidor, al no validar la entrada, resolverá la ruta como /etc/passwd, un archivo crítico que contiene la lista de usuarios del sistema.

Contramedidas:

- Evitar el uso de nombres de archivo proporcionados por el usuario: Usar IDs numéricos en su lugar.
- Validación por Lista Blanca: Solo permitir nombres de archivos específicos predefinidos.
- Sanitización con `basename()`: Función que extrae únicamente el nombre del archivo, eliminando cualquier ruta de directorio.
- Validación de Extensiones: Asegurar que el archivo solicitado tenga la extensión esperada (ej. .jpg, .png).
- Configuración del Servidor: Almacenar los archivos fuera del directorio raíz web y usar permisos de sistema de archivos restrictivos.

Documentación técnica de laboratorio

El laboratorio presenta una aplicación de comercio electrónico que carga imágenes de productos a través de una URL predecible: `GET /image?filename=imagen.jpg`. El código del servidor backend toma el valor del parámetro `filename` y lo concatena directamente a una ruta base en el sistema de archivos (ej. `/var/www/images/`) sin aplicar ninguna validación o sanitización previa. Esta falta de control permite que un atacante inserte secuencias de navegación (`../`) para salir del directorio de imágenes y acceder a cualquier archivo del sistema.

La petición original observada en Burp Suite es:

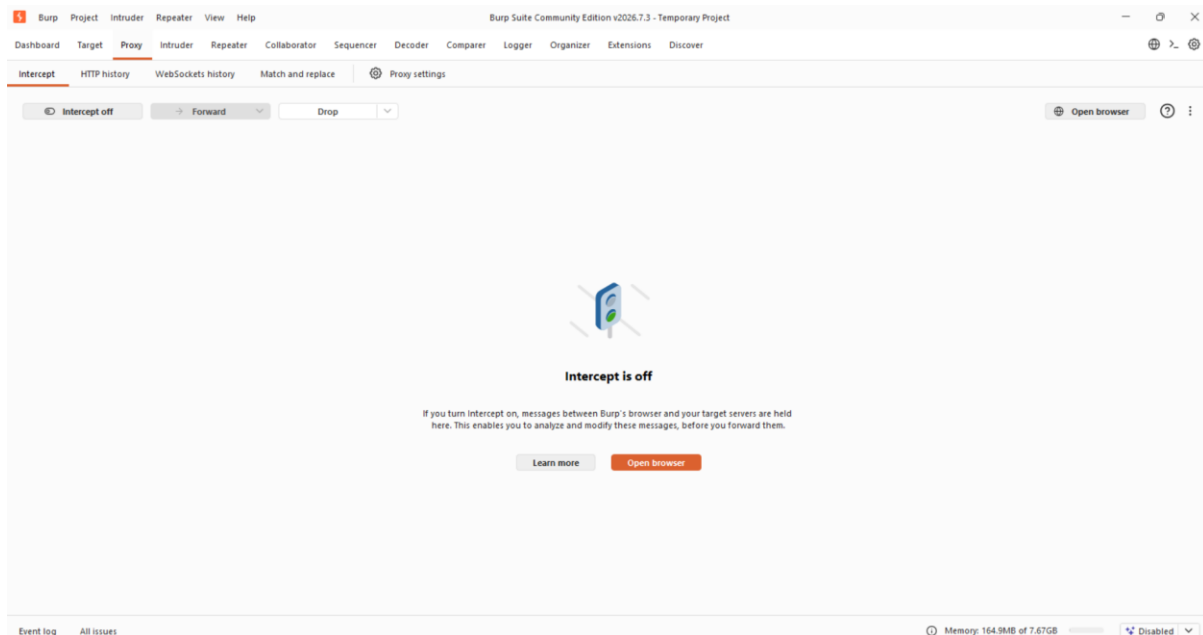
```
GET /image?filename=21.jpg HTTP/1.1
```

El parámetro `filename` es el punto de entrada vulnerable. El objetivo del laboratorio es leer el archivo `/etc/passwd` del servidor. La alteración de la consulta consiste en reemplazar el nombre de la imagen legítima por una carga útil (payload) de traversía. La petición maliciosa queda de la siguiente manera:

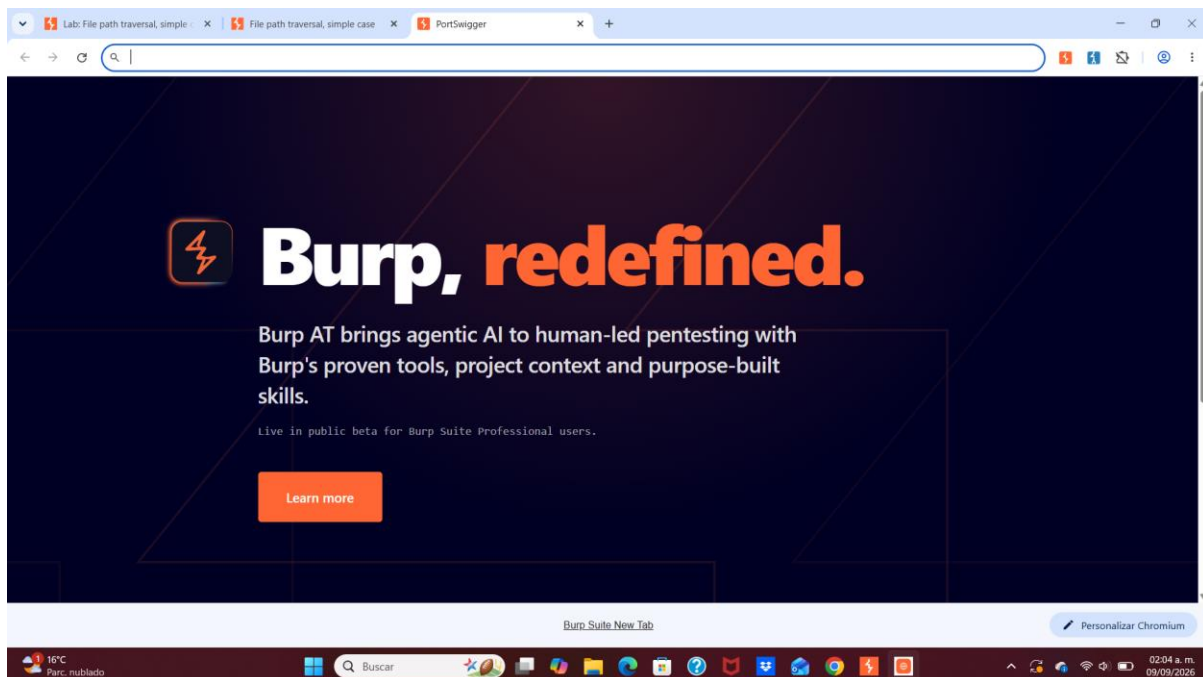
```
GET /image?filename=../../etc/passwd HTTP/1.1
```

Aquí, la secuencia `../../` le indica al sistema de archivos que suba tres niveles desde el directorio actual (`/var/www/images/`) hasta la raíz (`/`), para luego acceder al directorio `etc` y leer el archivo `passwd`. La petición mantiene todas las cabeceras originales (Host, Cookie de sesión, User-Agent), lo que demuestra que el ataque se realiza dentro del contexto autenticado de la aplicación.

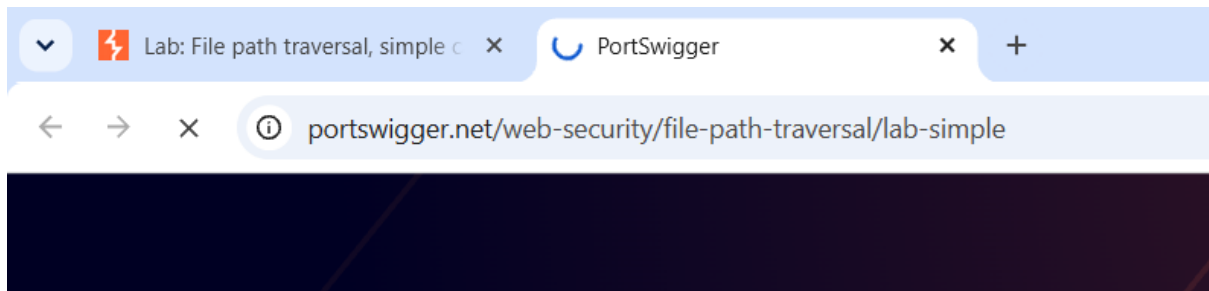
Desarrollo



Abrimos nuestra aplicacion de Burp del escritorio



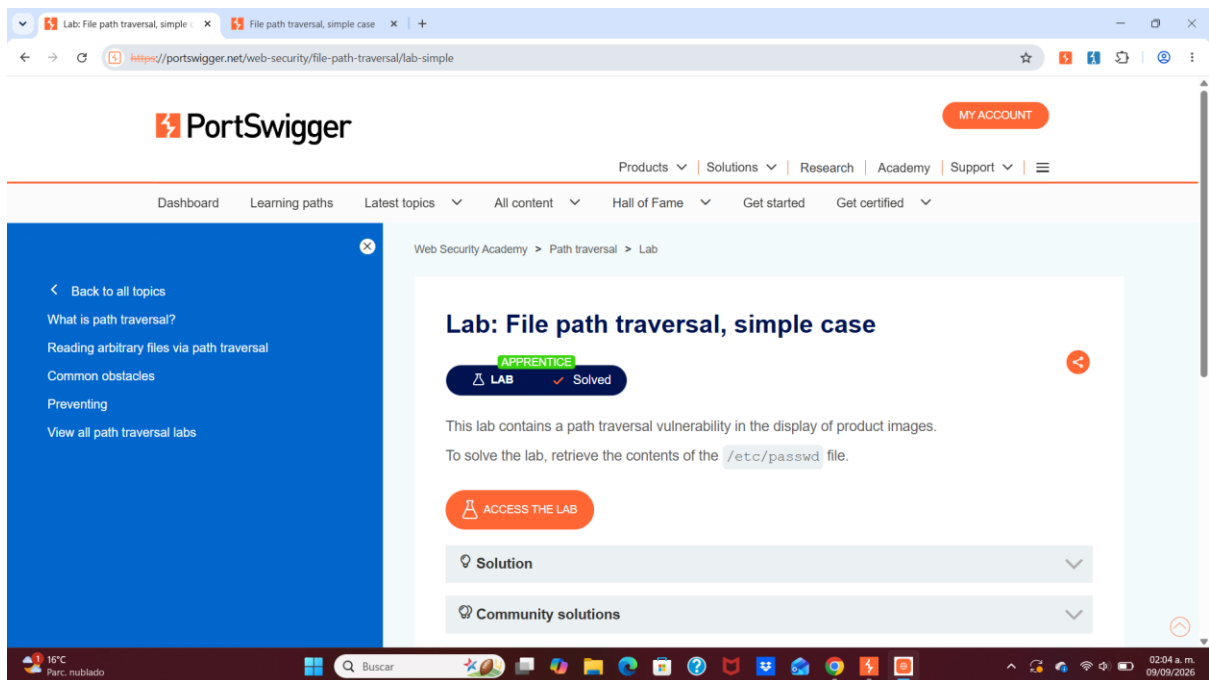
Abrimos el navegador que nos proporciona el software

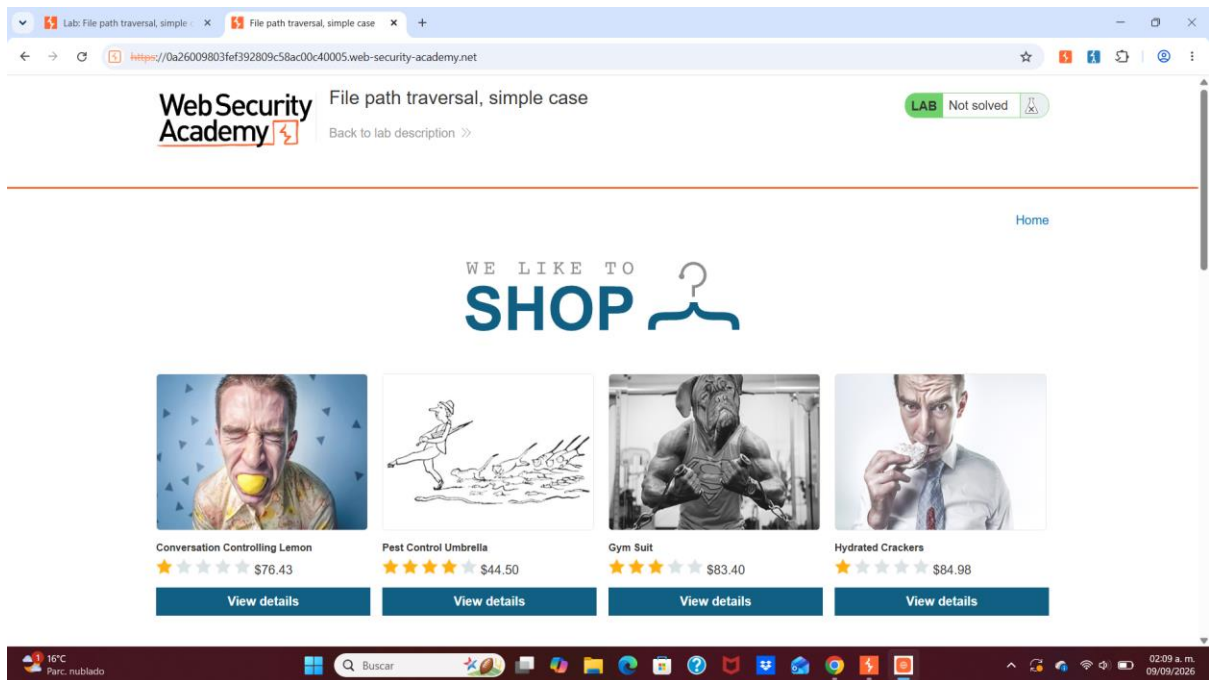


Buscamos la URL proporcionada para acceder a la actividad



Nos aparecerán estas dos respuestas, continuaremos dando Forward para que cargue la página correctamente. Previo a todo esto es obligatorio encender el botón de “Intercep” para que obtenga todo el tráfico, nosotros haremos caso solamente a las peticiones que nos interesan.





Enseguida de dar forward a las primeras dos peticiones, nos aparecerá la página completa, daremos click en cualquiera de las imágenes justo donde dice “View details”.

Time	Type	Direction	Method	URL	Status code	Length
02:09:29 9 Sep...	HTTP	→ Request	GET	https://0a26009803fef392809c58ac00c40005.web-security-academy.net/image?filename=7.jpg		
02:09:29 9 Sep...	HTTP	→ Request	GET	https://tags.srv.stackadapt.com/js_tracking?url=https%3A%2F%2Fportswigger.net%2Fweb-security%2Ffile-path-traversal%2Ffile-simple&uid=SEK-Q5_UK1F2mT5oZuJQ8v=1&host=htt...		
02:09:30 9 Sep...	HTTP	→ Request	GET	https://0a26009803fef392809c58ac00c40005.web-security-academy.net/academyLabHeader		
02:09:38 9 Sep...	HTTP	→ Request	POST	https://www.youtube.com/youtube/v1/log_event?alt=json		
02:09:38 9 Sep...	HTTP	→ Request	POST	https://www.youtube.com/youtube/v1/log_event?alt=json		
02:09:38 9 Sep...	HTTP	→ Request	POST	https://www.youtube.com/youtube/v1/log_event?alt=json		

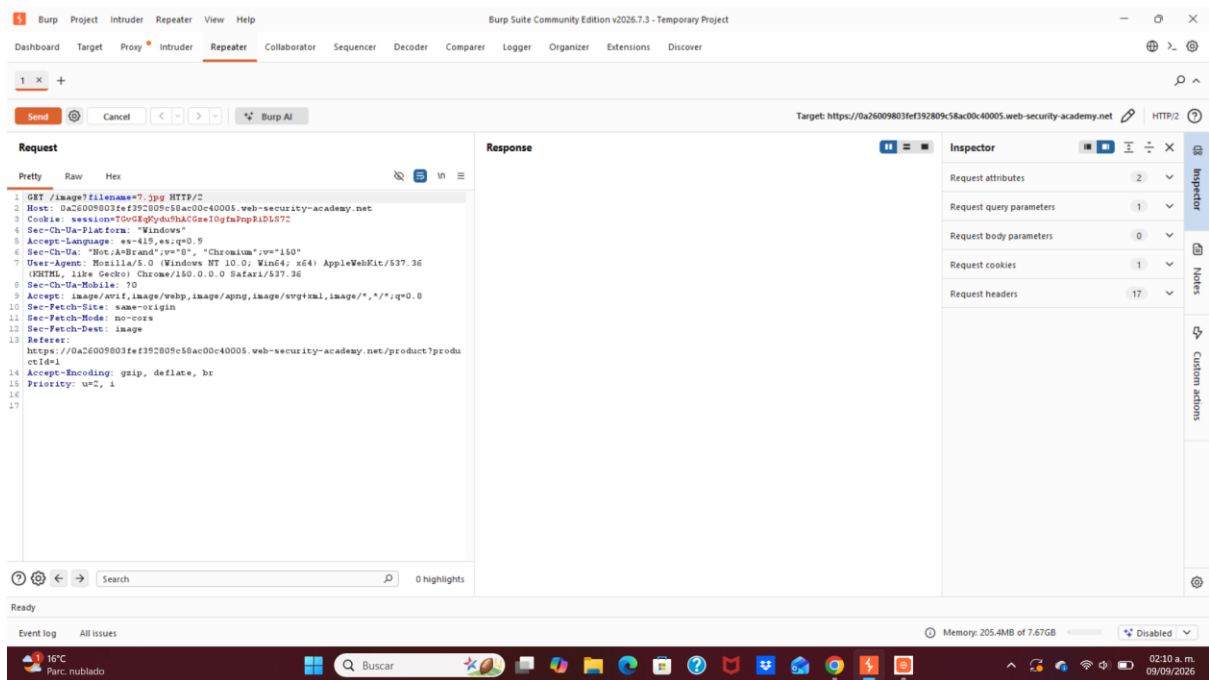
En la parte de Proxy/Intercept comenzaremos a ver que nos saldrán más peticiones daremos click en forward hasta llegar a las peticiones que en este caso nos importan que serán las de tipo “image”.

Time	Type	Direction	Method	URL
02:09:29 9 Sep...	HTTP	→ Request	GET	https://0a26009803fef392809c...demy.net/image?filename=7.jpg
02:09:29 9 Sep...	HTTP	→ Request	GET	https://0a26009803fef392809c...demy.net/image?filename=7.jpg
02:09:30 9 Sep...	HTTP	→ Request	GET	https://0a26009803fef392809c...demy.net/image?filename=7.jpg
02:09:38 9 Sep...	HTTP	→ Request	POST	https://0a26009803fef392809c...demy.net/image?filename=7.jpg
02:09:38 9 Sep...	HTTP	→ Request	POST	https://0a26009803fef392809c...demy.net/image?filename=7.jpg
02:09:38 9 Sep...	HTTP	→ Request	POST	https://0a26009803fef392809c...demy.net/image?filename=7.jpg

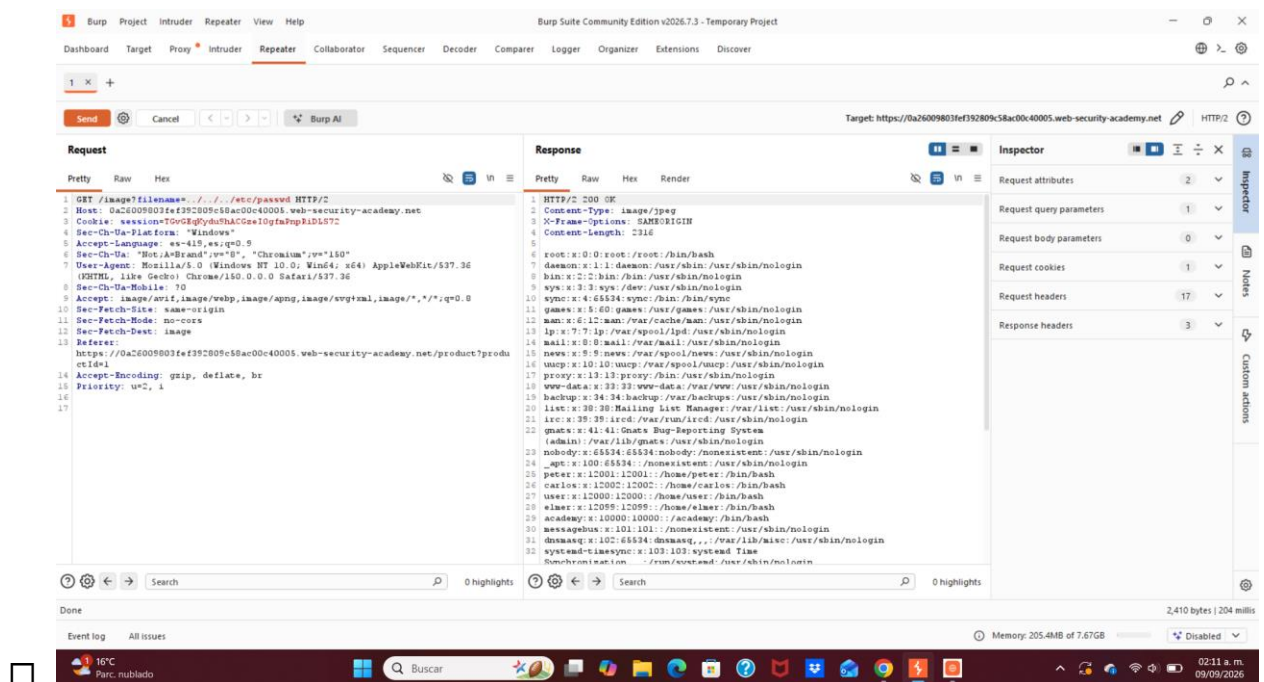
Request	Raw	Hex
---------	-----	-----

Add to scope Forward Drop Add notes Highlight Don't intercept requests Do intercept Scan Send to Intruder Send to Repeater Send to Sequencer Send to Organizer Send to Comparer Request in browser	Ctrl+I Ctrl+R Ctrl+O
---	----------------------------

Daremos click derecho y se nos desplegarán varias opciones, a nosotros nos interesa la que dice “Send to Repeater” que como su nombre lo dice es al Repetidor a donde mandaremos esta petición y donde la analizaremos.



Una vez estemos dentro de la sección del repetidor apreciaremos que se encuentra parte del código de la petición que estamos analizando nos centraremos en la parte que dice filename, que es por donde haremos nuestro ataque.



Una vez identifiquemos el filename borraremos lo que tiene por defecto lo que se supone tiene que mandar al servidor, y lo cambiaremos por “../../../../etc/passwd” lo que nos permitirá ver Usuarios, sus contraseñas encriptadas que se presentan con el carácter “x,” sus shell, sus ID de usuario como de grupo principal (UID y GID) así como su directorio principal. Información a la cual el usuario o en este caso el atacante no debería tener acceso.

Conclusión

La resolución de este laboratorio de File Path Traversal nos deja una enseñanza clara y contundente: la seguridad en el desarrollo web no es un añadido, sino una necesidad imperante desde la primera línea de código. La vulnerabilidad explotada, aunque clasificada como "simple", demuestra que un solo descuido en la validación de entradas puede abrir una brecha gigantesca en la seguridad de todo un sistema. El acceso no autorizado al archivo `/etc/passwd` no es solo la lectura de un archivo; es la confirmación de que un atacante ha superado las defensas perimetrales y ahora tiene información privilegiada que puede usar para escalar privilegios o preparar ataques más sofisticados.

El uso de herramientas como Burp Suite se revela no solo como un medio para explotar, sino como un instrumento de diagnóstico fundamental. Para un futuro profesional de la ciberseguridad, la capacidad de interceptar, analizar y modificar peticiones HTTP es la diferencia entre teorizar sobre una vulnerabilidad y comprenderla en su totalidad. Saber que la secuencia `../` es peligrosa es el primer paso; saber exactamente cómo, por qué y en qué contexto se ejecuta esa peligrosidad es lo que permite diseñar defensas efectivas.

En última instancia, este ejercicio refuerza el principio de "defensa en profundidad". No basta con una sola medida de seguridad. La combinación de una validación estricta de entradas, el uso de listas blancas, la sanitización con funciones como `basename()` y una correcta configuración de permisos en el servidor son capas que, juntas, hacen que el sistema sea resiliente. Comprender el ataque desde la perspectiva del atacante, usando las mismas herramientas, es la mejor manera de anticiparse y proteger los activos digitales en un mundo donde las amenazas son cada vez más complejas y constantes.